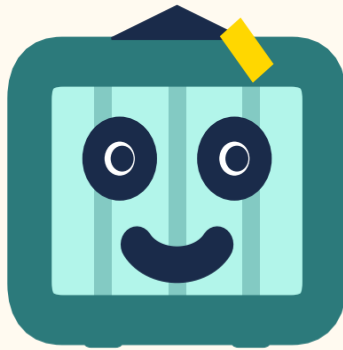


DATA STRUCTURES & ALGORITHMS

Trees, graphs, and the patterns behind LeetCode — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Big-O: How Algorithms Scale
2. Arrays vs. Linked Lists
3. Binary Search Tree
4. Four Ways to Walk a Tree
5. Graphs: Two Representations
6. BFS vs. DFS
7. Two Pointers and Sliding Window
8. Dynamic Programming
9. Heaps
10. Cheat Sheet

Most LeetCode problems are really a small set of underlying patterns wearing different costumes. This guide builds the pattern-recognition instinct: given a new problem, which of these shapes does it actually match?

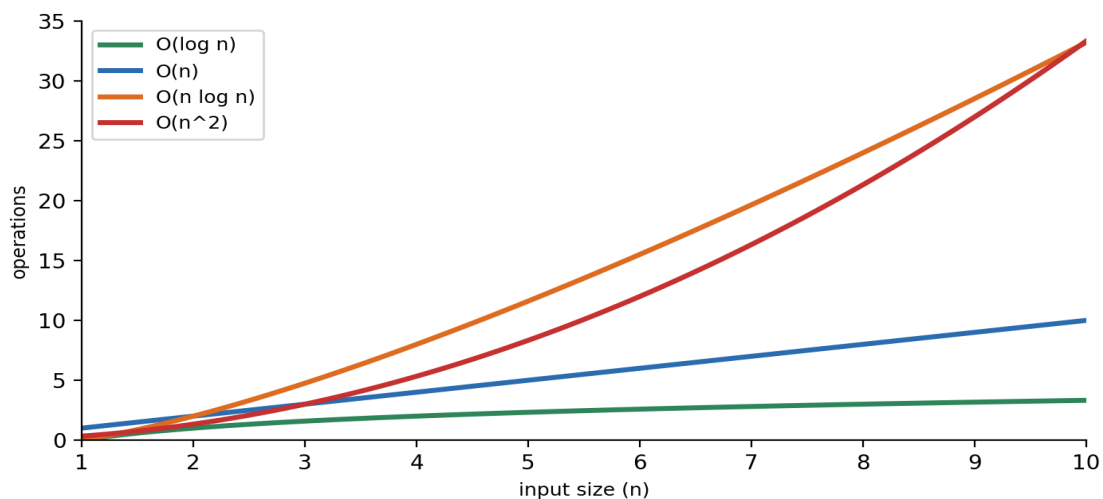
CHAPTER 1

Big-O: How Algorithms Scale

Big-O describes how an algorithm's work grows as input size grows -- not the exact runtime, but the shape of the curve. That shape is what determines whether an algorithm is usable at real-world scale or not.

The gap between $O(n \log n)$ and $O(n^2)$ looks small for tiny inputs, but explodes as n grows -- which is exactly why an $O(n^2)$ nested-loop solution that passes on a small test case can time out completely on a large one.

Big-O: How Algorithms Scale



The gap between $O(n \log n)$ and $O(n^2)$ is exactly why sorting algorithm choice matters enormously at scale.

IN SHORT: Always ask: does this solution's complexity actually scale to the input sizes the problem promises? A correct-but-too-slow solution is still a wrong answer.

CHAPTER 2

Arrays vs. Linked Lists

An array's elements sit in one contiguous block of memory -- which is exactly what makes index-based access $O(1)$: the address of element i is just a simple calculation from the base address. Inserting in the middle, though, means shifting everything after it.

A linked list's nodes are scattered in memory, each pointing to the next. Inserting or removing a node (given a reference to it) is $O(1)$ -- no shifting needed -- but reaching the k -th element means walking from the head, one node at a time.

Arrays vs. Linked Lists

Array: contiguous memory



$O(1)$ random access
 $O(n)$ insert in the middle

Linked List: scattered nodes



$O(1)$ insert/remove (with a reference)
 $O(n)$ random access

IN SHORT: Need fast random access? Array. Need fast insertion/removal at a known position? Linked list. Most real interview problems actually want the array.

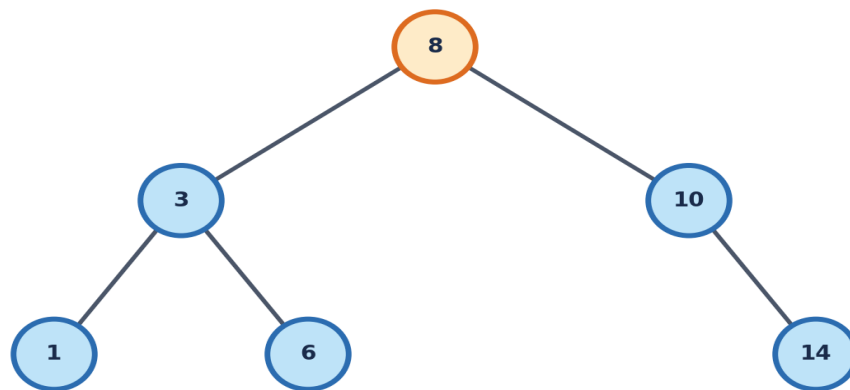
CHAPTER 3

Binary Search Tree

A BST maintains one simple invariant at every single node: everything in the left subtree is smaller, everything in the right subtree is larger. That's the entire rule, applied recursively.

This invariant is what makes search, insert, and delete all $O(\log n)$ on average -- each comparison eliminates roughly half the remaining tree, the same way binary search does on a sorted array. But an unbalanced BST (e.g., built by inserting already-sorted data) degenerates into a plain linked list, and every operation becomes $O(n)$.

Binary Search Tree



*Left subtree < node < right subtree, at every node.
Search, insert, delete: $O(\log n)$ if balanced, $O(n)$ if it degenerates into a list.*

IN SHORT: The BST invariant (left < node < right, everywhere) is what enables $O(\log n)$ operations -- but only if the tree stays reasonably balanced.

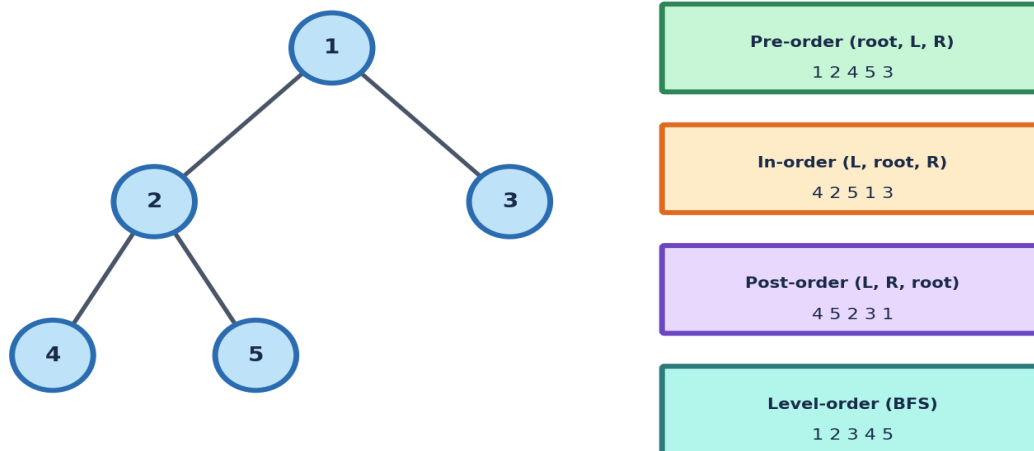
CHAPTER 4

Four Ways to Walk a Tree

Pre-order, in-order, and post-order are all depth-first traversals -- they differ only in when they visit the current node relative to its children. Pre-order visits the node first (useful for copying/serializing a tree). Post-order visits it last (useful for safely deleting a tree bottom-up).

In-order is special for a BST specifically: visiting left, then node, then right produces values in fully sorted order -- a direct consequence of the BST invariant from the previous chapter. Level-order is different in kind entirely -- it's breadth-first, using a queue instead of recursion, visiting the tree one full depth level at a time.

Four Ways to Walk a Tree



IN SHORT: In-order on a BST gives you sorted output for free. Level-order is the one that's BFS, not DFS -- and the one you reach for when "level by level" matters.

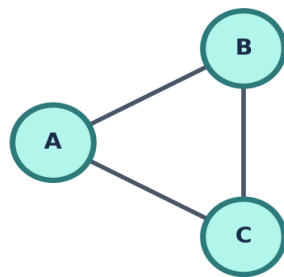
CHAPTER 5

Graphs: Two Representations

An adjacency list stores, for each node, just the list of its actual neighbors -- compact and efficient for sparse graphs (most real-world and most interview graphs, where each node connects to only a few others).

An adjacency matrix stores a full V-by-V grid, with a 1 or 0 indicating whether an edge exists between every possible pair of nodes -- $O(1)$ to check if a specific edge exists, but $O(V^2)$ space regardless of how few edges actually exist.

Graphs: Two Representations



The graph

Adjacency List	
A:	[B, C]
B:	[A, C]
C:	[A, B]

compact for sparse graphs (most graphs)

Adjacency Matrix			
	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0

$O(1)$ edge lookup, $O(V^2)$ space

IN SHORT: Default to an adjacency list unless you specifically need $O(1)$ "does this exact edge exist" lookups -- it's more space-efficient for the sparse graphs most problems actually give you.

CHAPTER 6

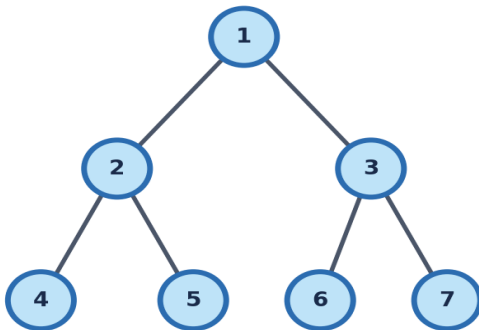
BFS vs. DFS

BFS explores level by level, using a queue -- visiting every neighbor of the current node before moving deeper. This is exactly why BFS is the standard choice for shortest path in an unweighted graph: the first time you reach a target node, you've necessarily reached it by the fewest possible edges.

DFS explores as deep as possible down one path before backtracking, typically via recursion or an explicit stack. It's the natural fit for problems about exploring every possible path, detecting cycles, or working through a graph/tree structurally rather than needing the shortest route.

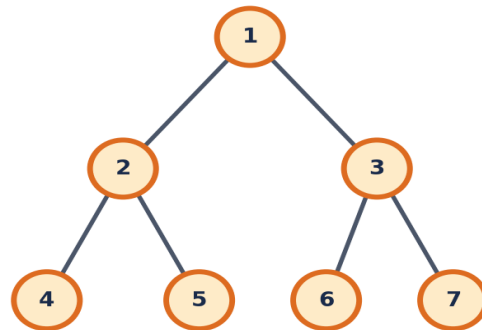
BFS vs. DFS

BFS: level by level



visit order: 1 2 3 4 5 6 7
(queue) -- shortest path

DFS: deep first



visit order: 1 2 4 5 3 6 7
(stack/recursion) -- explore fully

IN SHORT: Need the shortest path (unweighted)? BFS. Need to explore every path, or the problem is really about tree/graph structure? DFS.

CHAPTER 7

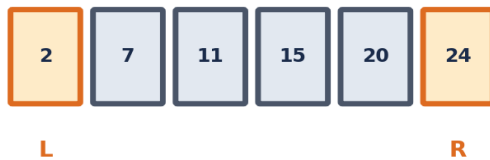
Two Pointers and Sliding Window

Two pointers is the classic technique for a sorted array where you need to find a pair (or more) satisfying some condition -- start one pointer at each end and move them toward each other based on how the current pair compares to the target, turning an $O(n^2)$ brute-force pair search into $O(n)$.

Sliding window applies the same "avoid recomputation" instinct to contiguous subarrays or substrings -- instead of recalculating a sum/count from scratch for every possible window, slide the window by one position and incrementally update, again turning an $O(n^2)$ or worse brute force into $O(n)$.

Two Pointers and Sliding Window

Two pointers (sorted array)



*converge from both ends:
 $O(n)$ instead of $O(n^2)$*

Sliding window (subarray)



*slide one step, update
incrementally: $O(n)$ total*

IN SHORT: Both patterns exist to eliminate redundant work a naive nested loop would repeat. Sorted array + pair/triplet -> two pointers. Contiguous subarray/substring -> sliding window.

CHAPTER 8

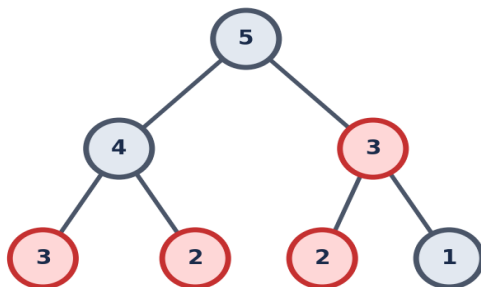
Dynamic Programming

A naive recursive solution to a problem like Fibonacci recomputes the exact same subproblems over and over -- $\text{fib}(3)$ gets computed independently every time it's needed higher up the call tree, and the total work grows exponentially.

Dynamic programming's entire trick is recognizing these overlapping subproblems and computing each one exactly once, storing ("memoizing") the result for instant reuse -- turning exponential work into linear or polynomial work, at the cost of some extra memory to store those cached results.

Dynamic Programming: Remember, Don't Repeat

Naive $\text{fib}(5)$



Same subproblems (red)
recomputed again and again.
Exponential.

With memoization



each subproblem solved *ONCE*,
stored, and reused: $O(n)$



IN SHORT: See a recursive solution repeatedly solving identical smaller subproblems? That's the signal for DP -- memoize, and an exponential solution often becomes linear.

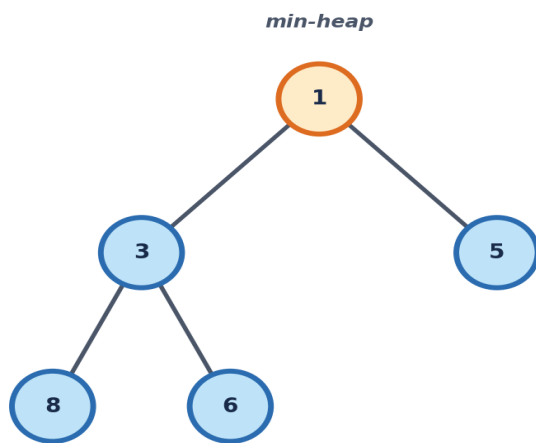
CHAPTER 9

Heaps

A min-heap keeps the smallest element always at the root, accessible in $O(1)$ -- while push and pop are both $O(\log n)$, far better than the $O(n)$ a sorted list would need to maintain the same always-know-the-minimum property under repeated insertions.

This makes a heap the natural tool for any "top-K" or "Kth largest/smallest" problem: maintain a heap of exactly size K, and for each new element, push it in and pop the smallest if the heap now exceeds size K -- what remains at the end is exactly your top-K set.

Heaps: Always Know the Min (or Max)



Top-K pattern

Keep a min-heap of size K.
Each new element: push,
then pop the smallest
if size > K.

peek: $O(1)$ push/pop: $O(\log n)$

IN SHORT: "Kth largest," "top-K," "closest K points" -- all the same underlying pattern: a size-bounded heap, doing $O(\log n)$ work per element instead of a full $O(n \log n)$ sort.

CHAPTER 10

Cheat Sheet

The pattern-recognition map: given a problem, which tool fits?

DSA PATTERN CHEAT SHEET



Recognize -> Reach for

Sorted array + target pair -> Two Pointers

Subarray/substring, contiguous -> Sliding Window

Shortest path, unweighted -> BFS

All paths / combinations -> DFS + Backtracking

More patterns

Overlapping subproblems -> DP (memoize)

Top-K / Kth largest -> Heap

Fast lookup by value -> HashMap/HashSet

Range queries, sorted data -> Binary Search

Complexities to know

Array access: $O(1)$ | search: $O(n)$

Sorted array binary search: $O(\log n)$

Hash map get/put: $O(1)$ average

Balanced BST ops: $O(\log n)$

Heap push/pop: $O(\log n)$

Tree traversal uses

In-order on BST -> sorted output

Level-order -> shortest tree path, by level

Post-order -> delete/free nodes safely

Pre-order -> copy/serialize a tree